

Parked, Not Idle: Why Autoscaling Breaks on Agentic LLM Workloads

Anonymous Author(s)
Submission #xxx

Abstract

Every LLM autoscaler in production — Kubernetes HPA on GPU utilisation, KEDA on queue depth, llm-d’s KV-cache-driven Workload Variant Autoscaler — inherits an assumption from stateless web serving: a request that is not computing is not occupying the system. It has either finished, and released its resources, or it is queued, and therefore visible.

Agentic workloads violate this. An agent program is a sequence of model invocations separated by tool executions, and during a tool call the program consumes no compute while holding its entire accumulated KV cache resident. It is *parked*, not idle: invisible to every compute-side signal, yet fully occupying memory.

We quantify the consequences with a trace-driven study built on the public Azure LLM inference traces. Azure’s coding workload, the closest production analogue of an agent, differs from chat in kind rather than degree: index of dispersion 71.3 against 4.0, peak-to-mean 12.8 against 1.8, context-to-generation ratio 73:1 against 5.5:1. Simulating an elastic cluster, we find that at eight turns per program **48.6% of resident programs are parked** at any instant.

The effect on autoscaling is severe. Across the agentic region of our sweep the deployed policies attain 0.25–0.37 of the SLO and complete 0.17–0.31 of offered programs, against 0.66–0.99 SLO on the same trace replayed as chat. We evaluate six policies spanning all three branches of the autoscaling taxonomy, including a proactive forecaster and a tabular Q-learner. The Q-learner is the worst of all, completing *zero* programs, because a 180-second cold start makes the credit assignment between an immediate cost saving and a delayed SLO collapse essentially unlearnable.

PARKAWARE changes one thing — occupancy counts parked programs — and attains 0.655 SLO and 0.527 goodput, 1.46× and 1.28× the best baseline. Its margin is a direct function of how expensive state is to rebuild: at a 30s cold start it leads by 1.04×, at a realistic 180s for a 70B model by 2.48× over HPA.

Keywords

LLM serving, agents, autoscaling, elasticity, KV cache

1 Introduction

An autoscaler answers one question: how many replicas should be running right now? For two decades the answer came from a utilisation signal, and underneath every such signal lies the same assumption. Work that is not being executed is not present. It has finished and released its resources, or it is waiting and shows up in a queue. Either way, a utilisation reading of zero means an empty machine.

LLM agents falsify this. An agent program is not a request; it is a loop. The model emits an action, an external tool executes it, the result is appended to the transcript, and the model is invoked again on the extended context [2]. Between invocations the program computes nothing. It also releases nothing: its KV cache, now holding the whole transcript, stays resident, because evicting it means re-prefilling tens of thousands of tokens when the tool returns. The program is *parked*.

A parked program is invisible to every compute-side signal an autoscaler can read. `DCGM_FI_DEV_GPU_UTIL` reports SM occupancy and sees nothing, because no kernel runs on its behalf. `vllm:num_requests_waiting` counts requests queued for a replica and sees nothing, because the program is not waiting for a replica — it is waiting for a shell command to return. To an autoscaler reading those signals, a cluster full of parked agents looks like a cluster that should be shrunk.

Shrinking it is worse than merely wrong. Removing a replica destroys the KV cache of every program parked on it, and each of those programs must re-prefill its full transcript when its tool returns. That recompute lands on a cluster that was just made smaller, driving latency up, which the autoscaler eventually notices — and then pays a cold start of minutes to undo a decision it made in seconds. The asymmetry between how fast you can destroy state and how slowly you can rebuild it is the whole problem.

This paper. We ask what happens to autoscaling when the workload becomes agentic, and answer with a trace-driven simulation study.

- **Characterisation.** Using the public Azure LLM inference traces [1], we show the coding workload differs from chat in kind: 18× the arrival burstiness and 13× the context-to-generation ratio. The workload that most needs an autoscaler is the one on which autoscalers fail (§2).
- **Quantifying parking.** At eight turns per program, 48.6% of resident programs are inside a tool call at any instant, holding memory and consuming no compute (§5.1).
- **A broad evaluation.** We implement six autoscalers spanning all three branches of the standard taxonomy — reactive (HPA, KEDA, KV-util), proactive (Holt forecasting), and learning-based (tabular Q-learning, with and without the parked signal) — and show all of them degrade sharply, in different ways (§5.2).
- **A negative result on RL.** The Q-learner is the worst policy we tested, completing zero programs. The failure is structural, not a tuning artefact: cost

is immediate and legible in the reward, SLO damage is delayed by a full cold start, and the agent reliably learns to starve the cluster (§5.4).

- **A mechanism, and evidence that it is the mechanism.** PARKAWARE counts parked programs as occupancy and recovers most of the loss. Its advantage is a monotone function of cold-start cost, vanishing when rebuilding state is cheap — which is what we would expect if, and only if, the mechanism we claim is the one operating (§5.5).

Scope. This is simulation driven by real production arrival traces. We did not run vLLM on GPUs, and §6 states precisely what that does and does not license us to claim.

2 Background and Motivation

2.1 Agentic serving

A chat request is a function call: prompt in, completion out, state discarded. An agent program is a stateful loop. In the ReAct pattern [2] that underlies most deployed agents, the model alternates between reasoning steps and tool invocations, and every tool result is appended to a transcript resent on the next invocation. Two properties follow.

Context accumulates. Turn i 's prompt contains everything from turns $0 \dots i-1$. A program that runs long enough has a prompt of tens of thousands of tokens, almost all of it prefix already computed on a previous turn — which is exactly why serving systems work so hard to keep it cached [3].

Programs are long. Published measurements of real coding-agent trajectories report an average of 51 assistant turns per session, with a tail beyond 200 [10]. The turn counts we sweep, up to 64, are therefore at or below what production agents actually do.

2.2 Autoscaling signals

Three signals are used in practice. *GPU utilisation* is what Kubernetes HPA reads by default in every GPU-autoscaling guide. *Queue depth* — `vllm:num.requests.waiting` — is what the vLLM production documentation and llm-d guides recommend for KEDA. *KV-cache utilisation* is the newest, used by llm-d's Workload Variant Autoscaler and named by HeteroScale as its planned direction.

The first two are compute-side; a parked program registers on neither. The third is memory-side and does see parked programs — but as undifferentiated pressure, without knowing they will return. As we show, it performs worst of the three.

2.3 Two Azure traces, two different workloads

Microsoft released two LLM inference traces from Azure production services, both November 2023: a *conversation* service and a *coding* service [1]. Each record has an arrival timestamp, a context length, and a generated length. The coding trace is the closest public proxy for agentic serving:

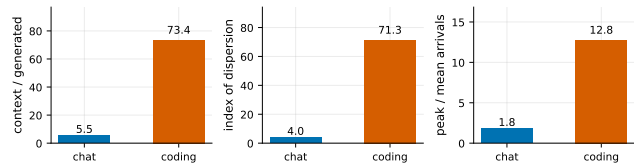


Figure 1: The two Azure traces differ in kind, not degree. Measured over 8,819 coding and 19,366 chat requests spanning about one hour each.

an assistant repeatedly sends an accumulated buffer and receives a short completion.

Figure 1 reports three measured differences.

Shape of the work. Coding has a mean context of 2,048 tokens and generates a mean of 27.9 — a ratio of 73:1. Chat is 1,155 and 211, a ratio of 5.5:1. Coding requests are almost pure prefill: the signature of a loop that accumulates state and emits a short action.

Burstiness. Binning arrivals into ten-second windows, coding has an index of dispersion of 71.3 against 4.0 for chat. Peak-to-mean is 12.8 against 1.8; the inter-arrival coefficient of variation is 13.2 against 1.09. Chat arrivals are nearly Poisson. Coding arrivals are emphatically not.

The second observation is the one that matters. An autoscaler exists to track load. A workload with peak-to-mean 1.8 hardly needs one; a workload with peak-to-mean 12.8 needs one badly. The workload that most requires autoscaling is precisely the one on which every deployed autoscaler fails.

3 The Parking Problem

3.1 Statement

Consider a cluster serving agent programs of T turns with mean tool time τ , and let s be the mean service time of one model invocation. A program's lifetime is $T(s + \tau)$, of which $T\tau$ is parked. In steady state with program arrival rate λ_p , Little's law gives $\lambda_p T(s + \tau)$ resident programs, of which a fraction

$$\phi = \frac{\tau}{s + \tau} \quad (1)$$

are parked at any instant. For a coding agent with $s \approx 1$ s and tool calls of a few seconds, ϕ exceeds one half.

Each parked program holds c_i tokens of KV, where c_i is its accumulated transcript. Total resident KV is $\sum_i c_i$ over running *and* parked programs, while compute demand counts only the running ones. The two quantities an autoscaler must reconcile diverge as $\phi \rightarrow 1$, and no compute-side metric can see the divergence.

3.2 Why scale-in is not the inverse of scale-out

For stateless serving, scaling in is the inverse of scaling out: remove a replica, drain or retry its in-flight requests, and the cost is bounded by one request's service time. Under agentic serving it is not. Removing a replica destroys the KV of

every program parked on it, and each pays a full re-prefill of c_i tokens on its next turn. The cost is

$$C_{\text{in}} = \sum_{i \in \text{parked}(\tau)} \frac{c_i}{R_{\text{prefill}}} + \mathcal{K}[\text{reversed}] \cdot t_{\text{cold}} \quad (2)$$

where the second term appears if the decision must later be undone. With c_i in the tens of thousands and t_{cold} at 180 s for a 70B-class model, one bad scale-in costs more than minutes of the replica it saved.

This asymmetry is why the problem is not solved by tuning a threshold. A symmetric controller — and all deployed ones are symmetric — treats adding and removing a replica as equal and opposite. Under agentic serving they are neither.

4 Method

4.1 What is real and what is constructed

The Azure traces predate agentic serving. They carry no program identifiers and no tool-call durations, and to our knowledge no public trace with production arrival times carries both. We state the boundary explicitly.

Real, from the trace: arrival timestamps, and therefore the entire arrival process including its burstiness; per-invocation context length; per-invocation generated length.

Constructed, and swept: which invocations compose a program (T turns), and the tool think-time between turns (τ).

We never fix T and τ and report one number. We sweep $T \in \{1, 4, 16, 64\}$ and $\tau \in \{0, 8, 30\}$ s and report the surface. $T=1, \tau=0$ recovers the original request-level trace exactly; it is chat, it is our control, and on it the baselines should work — and do.

The sweep range is calibrated to published measurements of real agent runs rather than chosen for convenience. Polar reports a mean of 51 assistant turns per SWE-agent session with a tail past 200 [10]; our $T=64$ therefore sits near the middle of the real distribution, and conclusions drawn at $T=16$ are conservative.

Context accumulates as in a real agent: turn i ’s prefill is the whole transcript so far, growing by what the model generated plus what the tool returned. Under this construction a 16-turn program reaches a mean final context of 8,510 tokens and a p99 of 34,340 — the right order of magnitude for deployed coding agents.

4.2 Simulator

We simulate a cluster of replicas, each with a KV capacity and a continuous-batching service model. Aggregate throughput saturates with batch size: with k concurrent turns each advances at rate $R/(k + k_{1/2})$, so per-turn latency degrades far more slowly than $1/k$. This is what batching buys, and a naive processor-sharing model that missed it would overstate the cost of load and flatter every policy that over-provisions.

Defaults model a 70B-class model on two A100-80GBs, the regime agentic deployments run in because agents need

strong tool-use models: 62,000 tokens of KV per replica (160 GB minus ~ 140 GB of weights at 320 KB per token), $R = 4,000$ token-equivalents/s, $k_{1/2} = 16$, batch ≤ 32 , and a 180 s cold start to load 140 GB of weights and warm up. Cold start is swept in §5.5.

A program loses its KV two ways: *eviction*, when its replica exceeds capacity, and *scale-in*, when the autoscaler removes the replica it is parked on. Either way the next turn re-prefills the whole transcript. We count the two separately, because attributing the damage is the point.

4.3 Metrics, and a survivorship trap

Turn service times span two orders of magnitude, so an absolute latency bound is meaningless. We use *slowdown*: a turn’s latency divided by what it would have taken alone on a replica. The SLO is $\text{slowdown} \leq 3$.

An early version of this study scored SLO attainment over turns that *completed*, and produced a striking result: our Q-learning autoscaler achieved perfect attainment. It had learned to shrink the cluster to one replica, whereupon 861 of 936 programs never finished, and the handful of short turns that did complete all met the SLO. The metric rewarded starvation.

We therefore count every turn that was admitted but never completed as a violation, and report *goodput* — the fraction of offered programs that finished — as a first-class metric alongside SLO. Every number in this paper uses the corrected definitions. We report the trap because it is easy to fall into and, where programs are long, it inverts the ranking of policies.

4.4 The fairness rule

Comparing autoscalers on cost alone is trivially gamed by scaling to one, and on SLO alone by scaling to the maximum. For every workload point we search for the cheapest *static* cluster meeting the SLO and use it as the reference: an autoscaler is interesting only if it holds the SLO for less than a static cluster would cost, because a static cluster is what you get for not autoscaling at all. All policies are warm-started at that size and the first 300 s are excluded, so we measure tracking rather than an arbitrary initial transient.

4.5 Policies

We implement six, spanning all three branches of the standard autoscaling taxonomy [9]. The baselines are what is deployed, not straw men.

Reactive. HPA (GPU util) is the Kubernetes autoscaler on `DCGM_FI_DEV_GPU_UTIL`, target 70%, with HPA’s exact recommendation formula and the default 300 s scale-down stabilisation window. **KEDA (queue)** scales on `vllm:num.requests.waiting`, the metric the vLLM production guides recommend. **KV-util** targets 80% KV-cache utilisation, as `llm-d`’s Workload Variant Autoscaler does.

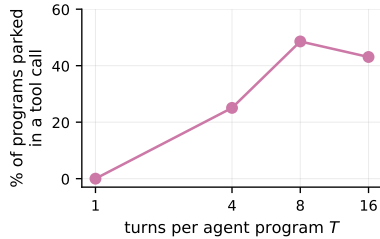


Figure 2: Fraction of resident programs parked in a tool call, measured on a fixed cluster so no autoscaler can confound it.

Proactive. **Predictive** runs Holt double-exponential smoothing on recent arrivals and provisions for the forecast one cold-start horizon ahead. It is a strong baseline precisely because it targets the symptom our results are full of: reacting too late.

Learning-based. **RL (Q-learning)** is tabular Q-learning over a discretised state — replica count, queue depth, GPU utilisation, KV utilisation — with actions $\{-2, -1, 0, +1, +2\}$ replicas and reward $-(\text{cost}) - \lambda(\text{SLO violations})$. This is the classic formulation in the cloud-autoscaling literature [8, 9]. It trains on a disjoint 1,200s prefix for 25 episodes and is evaluated on the suffix; we tuned λ over $\{12, 40, 120, 400\}$ and report the best. **RL + parked** is the same agent with the parked count added to its state: the ablation separating “a better algorithm” from “a better signal”.

Ours. PARKAWARE changes one thing: occupancy counts parked programs.

$$N = \max\left(\left\lceil \frac{\text{running} + \text{parked} + \text{queued}}{B\theta} \right\rceil, \left\lceil \frac{\text{KV used}}{K\theta} \right\rceil\right) \quad (3)$$

with batch B , per-replica KV K , and target θ . The second term makes memory a floor rather than an afterthought; the first counts a program waiting on a tool as what it is, a seat that is taken. Scale-in passes through a 60s stabilisation window, an order of magnitude shorter than HPA’s default, because the parked term already supplies the damping.

5 Evaluation

5.1 How much of the cluster is parked?

Figure 2 answers the question motivating everything else. At $T=1$ — chat — nothing is parked, by construction. At $T=4$, 25.1% of resident programs are inside a tool call at any instant; at $T=8$, 48.6%; at $T=16$, 43.1%. Roughly half the cluster’s occupancy is invisible to any compute-side metric, and Eq. (1) predicts the trend.

5.2 Every autoscaler degrades

Figure 3 and Table 1 are the main result.

On the control point — $T=1$, the unmodified trace, i.e. chat — the deployed policies are fine: KEDA attains 0.986, HPA 0.897. This matters, because it establishes that the

Table 1: Agentic region ($T \geq 4$, $\tau \geq 8$ s), averaged. Cost is GPU-seconds relative to the cheapest static cluster meeting the SLO.

Policy	SLO attain.	Goodput	Cost
HPA (GPU util)	0.374	0.308	0.31
KEDA (queue)	0.330	0.286	0.24
KV-util (llm-d)	0.253	0.165	0.23
Predictive (Holt)	0.449	0.413	0.28
RL (Q-learning)	0.115	0.000	0.05
PARKAWARE (ours)	0.655	0.527	0.40

baselines are correctly implemented and well suited to what they were designed for. Nothing in our simulator handicaps them.

Averaged over the agentic region, attainment collapses to 0.253–0.449 and goodput to 0.165–0.413. PARKAWARE holds 0.655 and 0.527: 1.46 $\times$ the best baseline on SLO and 1.28 $\times$ on goodput.

The cost column identifies the failure mode. Every baseline spends 0.05–0.31 of the cheapest static cluster’s GPU-seconds. They are not mis-tracking symmetrically; they are systematically *shrinking* the cluster, because that is what their signals tell them to do, and paying in SLO and goodput. PARKAWARE spends 0.40 — still less than static, but it is the only policy that does not read a parked agent as an empty seat.

KV-util deserves comment, being the newest and, on paper, the one that should see parked programs. It performs worst of the three reactive policies (0.253 SLO, 0.165 goodput). Seeing memory is not enough: KV utilisation conflates memory about to be reused with memory merely occupied, and a replica full of parked programs reads identically to one that is genuinely saturated.

5.3 Where the damage comes from

Figure 4 separates the two ways a program loses its cache. Averaged over $T \geq 4$, scale-in destroys the KV of 3,108 programs per run under KEDA and 422 under HPA, against 221 under PARKAWARE — an order of magnitude fewer. KEDA is worst precisely because its signal is the most badly mismatched: an empty queue during a wave of tool calls reads as an idle cluster, and it removes replicas full of parked state.

Each destroyed program pays a full transcript re-prefill on its next turn. That recompute lands on a cluster just made smaller, raising latency, which the autoscaler eventually sees — 180 seconds later, after a cold start. Equation (2) prices this loop; the figure measures it.

5.4 Why reinforcement learning fails here

The Q-learner is the worst policy we evaluated. It attains 0.115 SLO and completes *zero* programs across the agentic region. Adding the parked count to its state does not help (0.107 versus 0.115).

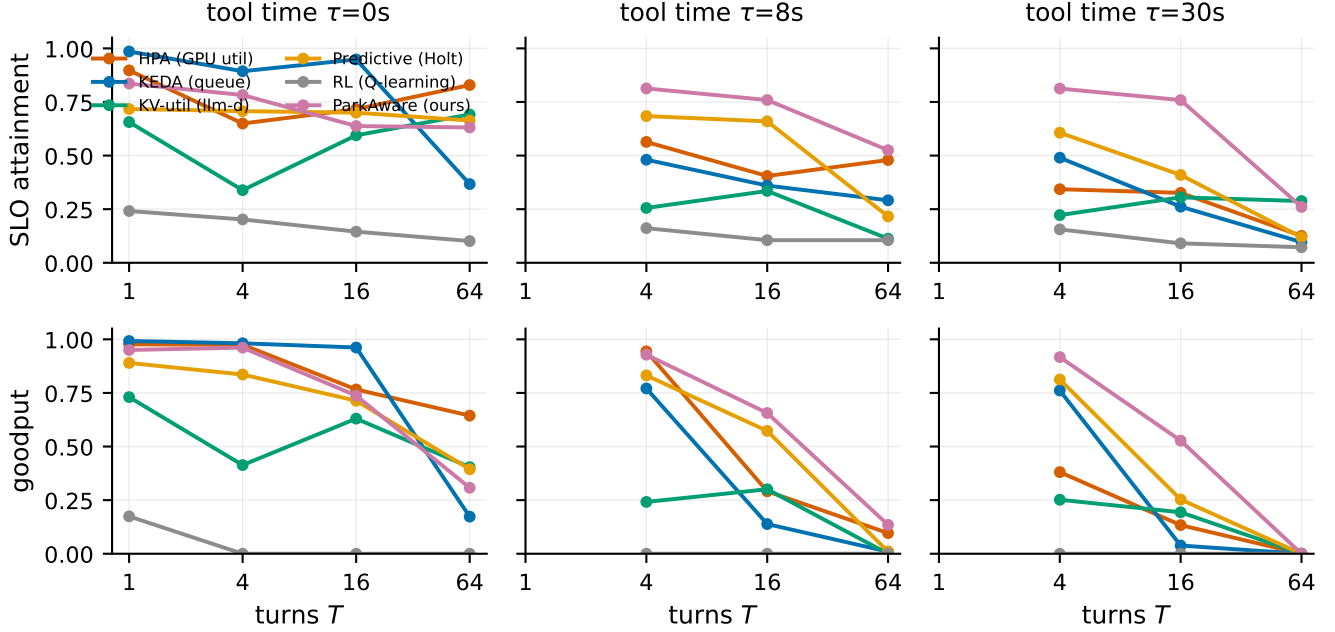


Figure 3: SLO attainment (top) and goodput (bottom) across the sweep. At $T=1$ the workload is chat and the deployed policies work. As programs lengthen and tool time grows every baseline collapses; ParkAware degrades most slowly. The Q-learner (grey) completes nothing anywhere.

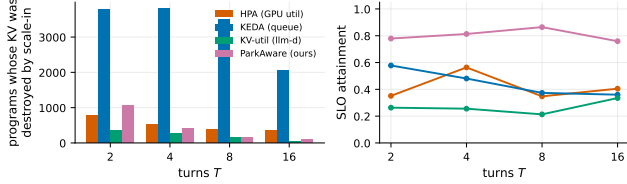


Figure 4: Left: programs whose KV cache was destroyed by a scale-in decision. Right: the resulting SLO attainment.

The failure is instructive and, we argue, structural rather than a tuning artefact. In the reward $-(\text{cost}) - \lambda(\text{violations})$, the cost term is immediate, dense, and exactly attributable to the action just taken. The violation term is delayed by up to a full cold start: remove a replica now, and the SLO damage appears 180 seconds and a dozen decisions later, by which time credit has diffused across the trajectory. The agent reliably learns the half of the reward it can see.

We tuned λ across a factor of 33 and the pathology persisted; at $\lambda=12$ it emerged in the most dramatic form, the one that motivated the metric correction in §4.3. We do not claim that no RL autoscaler can work — a method with a longer effective horizon, a shaped reward, or a learned

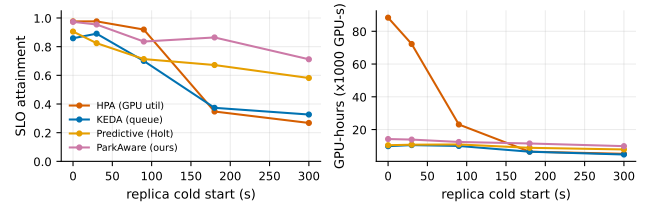


Figure 5: Sweeping replica cold-start time at $T=8, \tau=8s$.

model of cold start might well. We claim the obvious formulation, the one in the standard autoscaling literature, transfers badly to a setting where the dominant cost is delayed by minutes, and that this should be reported.

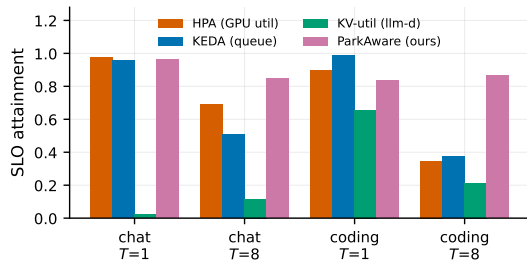
That the parked signal does not rescue it is itself informative. The two failures are independent: the baselines fail because their signal is blind, and the Q-learner fails because its credit assignment is broken. Fixing one does not fix the other.

5.5 The advantage is a function of cold start

Figure 5 and Table 2 isolate the mechanism. At a 30s cold start all policies are within a few percent: destroying state costs little when you can rebuild it quickly, so carelessness is nearly free. As cold start grows the baselines fall away

Table 2: SLO attainment against cold-start time ($T=8$, $\tau=8$ s).

Cold start	30 s	90 s	180 s	300 s
HPA	0.919	0.919	0.348	0.268
KEDA	0.891	0.700	0.373	0.327
Predictive	0.824	0.713	0.672	0.581
PARKAWARE	0.955	0.836	0.864	0.712
PARKAWARE/ HPA	1.04×	0.91×	2.48×	2.66×

**Figure 6: Both Azure traces, replayed as chat ($T=1$) and as agent programs ($T=8$, $\tau=8$ s).**

— HPA from 0.919 to 0.268, KEDA from 0.891 to 0.327 — while PARKAWARE degrades gracefully from 0.955 to 0.712.

This is the cleanest evidence that the mechanism we claim is the one operating. PARKAWARE is not a better forecaster; it makes no forecast at all. It simply declines to destroy state that is expensive to rebuild. Make rebuilding cheap and its advantage disappears, exactly as that account predicts.

The predictive baseline is the most interesting comparison. It is the best baseline at large cold start (0.672 at 180 s) because forecasting ahead is the textbook answer to slow provisioning — and it is still well behind PARKAWARE, because forecasting the *arrival* process says nothing about the programs already resident and parked. The two are complementary; combining them is future work we did not do.

5.6 The effect is not specific to the coding trace

Figure 6 repeats the experiment on the conversation trace. At $T=1$ all policies do well on both. At $T=8$ on the chat trace, HPA falls to 0.689 and KEDA to 0.506 while PARKAWARE holds 0.846; on the coding trace the same comparison is 0.348 and 0.373 against 0.864. The effect is a property of program structure, not of one trace, and it is worse on the burstier workload — which is the one that actually looks like an agent.

6 Threats to Validity

This is simulation. No number here was measured on a running GPU cluster. The service model, cold-start cost, and KV capacity are parameters chosen to match published hardware characteristics, not measurements we took. We

claim a mechanism and a direction, not a coefficient. The natural next step, which we have not done, is to replay these workloads against vLLM on real hardware; we designed the simulator’s metric interface to mirror vLLM’s exported Prometheus metrics so that substitution is mechanical.

Program structure is constructed. The composition of invocations into programs and the tool think-times are ours, not Azure’s, because no public trace with production arrival times carries them. We mitigate this three ways: sweeping both parameters over an order of magnitude and reporting the whole surface rather than a point; calibrating the sweep range to published statistics of real agent trajectories [10] rather than to convenience; and including $T=1$ as a control where the construction vanishes entirely and the baselines are seen to work. If real agent programs have a structure our sweep does not contain, our conclusions may not transfer.

The baselines are reimplementations. We implement HPA’s formula and stabilisation window, KEDA’s queue trigger, and llm-d’s KV-utilisation target from their documentation. Production deployments carry tuning we do not model, and a well-tuned KEDA might do better than ours.

Our RL agent may be too weak. Tabular Q-learning is the classic formulation, not the strongest possible. We report its failure and our diagnosis, and explicitly do not claim RL cannot work here.

ParkAware buys SLO with money. It spends 0.40 of the static reference while baselines spend 0.05–0.31. Whether that trade is right depends on a price the operator sets. Our evaluation assumes SLO and goodput are worth paying for; an operator with the opposite preference should read Table 1 in the other direction.

7 Related Work

Agent serving systems treat the agent program, rather than the request, as the scheduling unit. Autellix [4] schedules on cumulative program service time and reports large throughput gains over vanilla vLLM. Continuum pins KV across tool-call pauses using a predicted time-to-live. MORI places programs across HBM and DRAM by an idleness score. CONCUR bounds agent-level concurrency to prevent eviction thrashing. SAGA schedules whole workflows atomically. Every one optimises *within a cluster of fixed size*; none asks how large the cluster should be. That question is ours, and it is not answered by better scheduling: a perfectly scheduled cluster of the wrong size still misses its SLO.

LLM autoscaling is the complementary literature. TokenScale [5] scales disaggregated prefill and decode pools on token velocity. HeteroScale coordinates P/D ratios across heterogeneous hardware and names KV-aware autoscaling as future work. ServerlessLLM [6] attacks model-loading cold start directly, which §5.5 shows is the single most leveraged parameter. llm-d’s Workload Variant Autoscaler scales on KV utilisation. All target request-level chat serving. We show that the workload assumption, not the scaling mechanism, is what breaks.

Classical elasticity. Herbst et al. [7] defined cloud elasticity as the timeliness and accuracy of resource adaptation, and the taxonomy of reactive, proactive, and learning-based policies is due to the surveys [9]. The parked-program phenomenon is a case their framework does not cover: a unit of work that holds resources while contributing nothing to any utilisation signal. In their vocabulary, supply and demand are both mismeasured, in opposite directions, by the same metric.

RL for autoscaling has a long history [8, 9]. Our negative result is narrow — one formulation, one workload — but identifies a structural reason, that cost is immediate while SLO damage is delayed by a cold start, which we expect to generalise to any setting where provisioning latency greatly exceeds the control interval.

8 Conclusion

Autoscaling LLM clusters is a solved problem for chat and an open one for agents, and the reason is a single accounting error. A program waiting on a tool is not idle capacity; it is occupancy that no compute-side metric can see. Every deployed autoscaler makes this error. On agentic replays of production Azure traces it costs them roughly half their SLO attainment and two thirds of their goodput, and it costs a textbook RL autoscaler everything.

Counting parked programs recovers most of the loss, and does so by declining to destroy state rather than by predicting anything — which is why its advantage tracks the cost of rebuilding that state so precisely. We release the simulator, the workload builder, and every script that produced a number in this paper.

A Sensitivity

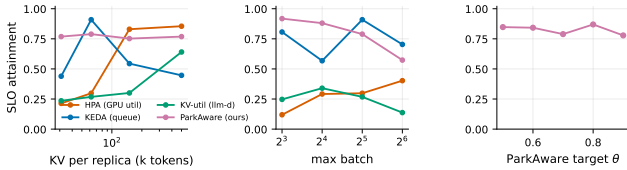


Figure 7: Sensitivity at $T=8, \tau=8$ s. Left: KV per replica, i.e. model size. Middle: maximum batch. Right: ParkAware’s target θ .

Figure 7 sweeps the three parameters most likely to be questioned.

Model size. Varying KV per replica from 31k tokens (a 140B model on two A100s) to 500k (an 8B model), PARKAWARE stays within a narrow band, 0.752–0.789, while the baselines swing violently: HPA between 0.214 and 0.855, KEDA between 0.439 and 0.909. The baselines are not merely worse on average; they are unpredictable, because whether their signal happens to correlate with true occupancy depends on where the binding constraint sits for that model size. PARKAWARE does not depend on that coincidence.

Batch size. PARKAWARE is strongest at small batches (0.918 at $B=8$) and weakest at large ones (0.573 at $B=64$). This is expected from Eq. (3): with a very large batch the compute term rarely binds and the policy reduces to its memory floor, which is KV-util with extra steps. Practical vLLM deployments of 70B-class models run at $B \leq 64$.

Target threshold. Across $\theta \in [0.5, 0.9]$ attainment stays between 0.778 and 0.869 with no monotone trend, so PARKAWARE does not require careful tuning — which matters, because an autoscaler that needs per-workload tuning is one that will be deployed with its defaults.

References

- [1] P. Patel et al. Splitwise: Efficient generative LLM inference using phase splitting. In *ISCA*, 2024. Traces: <https://github.com/Azure/AzurePublicDataset>.
- [2] S. Yao et al. ReAct: Synergizing reasoning and acting in language models. In *ICLR*, 2023.
- [3] W. Kwon et al. Efficient memory management for large language model serving with PagedAttention. In *SOSP*, 2023.
- [4] M. Luo et al. Autellix: An efficient serving engine for LLM agents as general programs. arXiv:2502.13965, 2025.
- [5] R. Lai et al. TokenScale: Timely and accurate autoscaling for disaggregated LLM serving with token velocity. arXiv:2512.03416, 2025.
- [6] Y. Fu et al. ServerlessLLM: Low-latency serverless inference for large language models. In *OSDI*, 2024.
- [7] N. Herbst, S. Kounev, R. Reussner. Elasticity in cloud computing: What it is, and what it is not. In *ICAC*, 2013.
- [8] E. Barrett, E. Howley, J. Duggan. Applying reinforcement learning towards automating resource allocation and application scalability in the cloud. *Concurrency and Computation*, 25(12), 2013.
- [9] Y. Garí et al. Reinforcement learning-based application autoscaling in the cloud: A survey. *Engineering Applications of Artificial Intelligence*, 2021.
- [10] Polar: Agentic RL on any harness at scale. arXiv:2605.24220, 2026. Reports 51 assistant turns per session on average with a tail beyond 200.
- [11] J. Stojkovic et al. DynamoLLM: Designing LLM inference clusters for performance and energy efficiency. In *HPCA*, 2025.
- [12] Y. Zhong et al. DistServe: Disaggregating prefill and decoding for goodput-optimized LLM serving. In *OSDI*, 2024.